AI Query Check

Evaluating the Correctness, Security, and Performance of LLM-Generated SQL

Deliverable 1: Benchmark Construction

Jelani Denmark (pipeline) Joshua Sanders (question set)
Isaiah Washington (formal methods & security)
Maxwell Amakor (schema & performance)

COSC 431 — Database Management
October 22, 2026

Abstract

This deliverable builds the benchmark that Deliverable 2 will use to evaluate LLM-generated SQL. We selected the Chinook digital-media-store schema, adapted 20 natural-language questions with expert-verified gold SQL from the Spider benchmark, executed and verified every gold answer against a local SQLite database, expressed five of the queries in relational algebra and tuple relational calculus, and constructed a deliberately denormalized schema variant for the performance comparison. All 20 queries execute without error and return non-empty results. Source repository: https://github.com/jd702/cosc431-ai-query-check

1 Submission checklist

Required item	Section	File in repository
Normalized schema (DDL)	App. A	`sql/normalized_schema.sql`
Denormalized variant + explanation	§5	`sql/denormalized_schema.sql`
20 questions + verified gold SQL	App. B	`data/questions.json`
and their result sets	App. B	`data/gold_results.json`
3–5 queries in relational algebra	§4	`docs/relational_algebra.md`
Write-up	§2–6	`docs/deliverable1_writeup.md`

Every artifact is reproducible from the repository with a single command, `python scripts/setup_db.py`, which downloads the Chinook SQL script, builds both database variants, and regenerates the DDL in Appendix A.

2 Schema choice

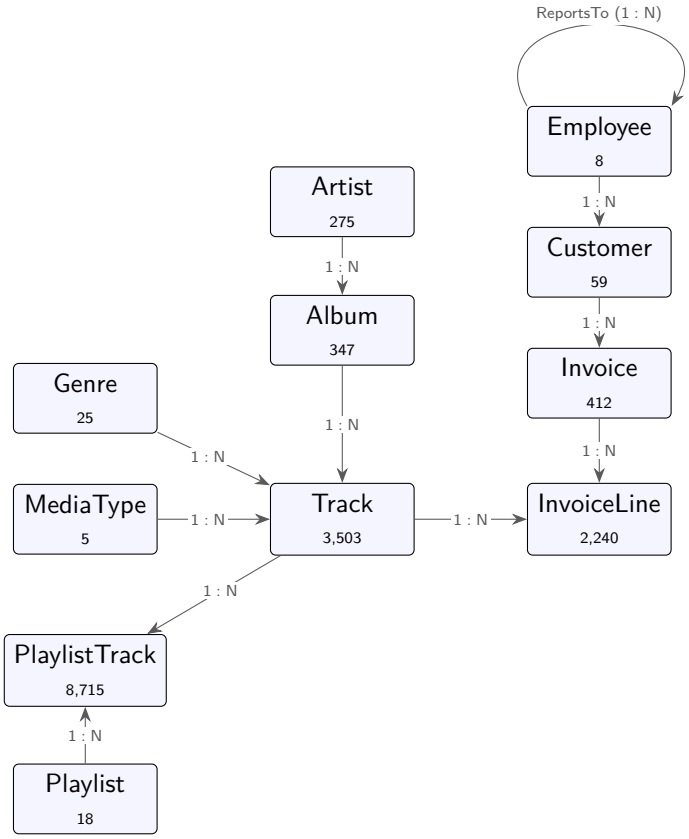
We chose **Chinook**, a digital media store, over Sakila. It is the smaller of the two options (11 tables against Sakila's 16), which let us spend our effort on the evaluation rather than on learning an unfamiliar schema, and it ships as a single SQLite script, so every team member can rebuild a byte-identical database. Chinook is also represented in Spider as `chinook_1`, so the questions we adapted were written against the same table and column names we query, which kept adaptations minimal.

Table	Rows	Role
Artist	275	Recording artists
Album	347	Albums, each belonging to one artist
Track	3,503	Individual tracks; the largest base table
Genre	25	Track genres
MediaType	5	Encoding formats
Playlist	18	Named playlists
PlaylistTrack	8,715	Many-to-many bridge, playlists to tracks
Customer	59	Customers, each assigned a support representative
Employee	8	Staff, with a self-referencing `ReportsTo` hierarchy
Invoice	412	Customer invoices
InvoiceLine	2,240	Line items on each invoice

The relationships give us the structural variety the project needs: simple one-to-many chains (Artist → Album → Track), a many-to-many bridge (PlaylistTrack), a self-referencing hierarchy (`Employee.ReportsTo`), and nullable foreign keys (`Track.AlbumId`, `Track.GenreId`, `Customer.SupportRepId`) that make join choice consequential. The Customer → Invoice → InvoiceLine → Track path is four tables deep, which is what makes the denormalization and indexing comparisons in Deliverable 2 meaningful.

Chinook as distributed is already in 3NF, so no normalization work was required to reach it. That is why this deliverable's normalization work runs in the opposite direction: we deliberately break 3NF to construct the denormalized variant in §5. Chinook also ships 11 foreign-key indexes of its own (named `IFK_*`), a detail that materially affects the Deliverable 2 performance method (§6).

2.1 ER diagram



Arrows point from the "one" side to the "many" side. `Track.AlbumId`, `Track.GenreId` and `Customer.SupportRepId` are nullable; all other foreign keys are mandatory. Full attribute lists are in Appendix A.

3 Question sources and adaptations

All 20 question/gold-SQL pairs were drawn from the Spider benchmark's `chinook_1` database (`train_spider.json`), not invented from scratch. We favored easy and medium difficulty as the project guide recommends, and selected for a mix of query shapes.

Difficulty	**Count**	**Query shape**	**Count**
Easy	6	Single-table lookup	7
Medium	14	Join (2–3 tables)	7
Hard	0	Aggregation (COUNT/SUM/AVG, GROUP BY)	5
		Subquery	1
Total	**20**	**Total**	**20**

Nine questions required adaptation. Three were changed in response to an internal review that found the question wording and the gold SQL disagreed — a mismatch that would have penalized a model for answering the question as written.

ID	Spider index	Change made
Q05	864	Reworded to "List the phone numbers of all employees" so the question matches the gold SQL, which projects only `Phone`.
Q08	822	Adapted to our table and column names.
Q11	849	None needed; Chinook uses identical table and column names.
Q12	870	Corrected the original gold SQL, which selected the *customer's* first name instead of the *employee's* and was grouped incorrectly.
Q13	837	Tightened the GROUP BY to include both `MediaTypeId` and `Name`.
Q15	840	Added `Title` to the GROUP BY so the album title is grouped with its id.
Q16	872	Changed INNER JOIN to LEFT JOIN so employees serving zero customers are not excluded, and clarified the question to say so.
Q17	888	Changed NOT IN to NOT EXISTS, avoiding SQL's three-valued-logic behavior when the inner result contains NULL.
Q20	888	Adapted the question and SQL to our schema.

Typographical errors present in the original Spider wording (for example "unite price", "do nout have") were deliberately left in place, to keep the questions faithful to the source dataset rather than silently improving the prompts the models receive.

3.1 Verification method

1. `notebooks/01_build_benchmark.ipynb` executes every gold query against `data/chinook.db` and writes the result sets to `data/gold_results.json`.
2. The notebook asserts that the set contains 15–20 questions, that IDs are unique, and that at least one question is a subquery.
3. Every result was reviewed by a team member other than the author of its SQL.
4. Selected answers were cross-checked independently against the raw data. For example, Q13's "least common media type" returns *Purchased AAC audio file* (7 tracks, fewer than *AAC audio file*'s 11), and Q17's count of artists without an album returns 71 (275 artists, of which 204 have at least one album).

All 20 queries execute with no SQL errors and no empty result sets.

4 Formal query notation

Five queries are expressed below in relational algebra, one additionally in tuple relational calculus. Each uses the verified gold SQL verbatim, so the identifiers match Appendix B. Operators: σ selection, π projection, $\bowtie$ join, γ grouping and aggregation, $-$ set difference.

Q06 — Invoice dates for the customer Astrid Gruber

```
SELECT T2.InvoiceDate FROM CUSTOMER AS T1
JOIN INVOICE AS T2 ON T1.CustomerId = T2.CustomerId
WHERE T1.FirstName = 'Astrid' AND T1.LastName = 'Gruber'
```

$$\pi_{InvoiceDate}\big(\sigma_{FirstName='Astrid' \wedge LastName='Gruber'}(\text{Customer} \bowtie_{\text{CustomerId}} \text{Invoice})\big)$$

An equivalent form pushes the selection below the join, which is exactly the rewrite a query optimizer performs:

$$\pi_{InvoiceDate}\big(\sigma_{FirstName='Astrid' \wedge LastName='Gruber'}(\text{Customer}) \bowtie_{\text{CustomerId}} \text{Invoice}\big)$$

Comparison. The algebra makes evaluation *order* explicit: the first form filters after materializing the join, the second filters first. Both denote the same relation, which is precisely why the rewrite is legal. The SQL states no order at all; it declares the result and delegates the plan to the engine, and `EXPLAIN QUERY PLAN` confirms SQLite chooses the second form. Conversely, the algebra requires an explicit theta-join condition $\bowtie_{\text{CustomerId}}$, because a natural join would silently equate every identically named column the two relations share — a hazard the SQL's explicit ON clause avoids.

Q10 — Count tracks of media type 'AAC audio file'

```
SELECT COUNT(*) FROM MEDIATYPE AS T1
JOIN TRACK AS T2 ON T1.MediaTypeId = T2.MediaTypeId
WHERE T1.Name = 'AAC audio file'
```

$$\gamma_{\text{COUNT}(TrackId) \to TrackCount}\big(\sigma_{Name='\text{AAC audio file}'}(\text{MediaType} \bowtie_{\text{MediaTypeId}} \text{Track})\big)$$

Comparison. `COUNT(*)` counts *rows*, including duplicates and all-NULL rows. The algebra forces a decision the SQL leaves implicit: we wrote COUNT($TrackId$), which counts non-NULL values of a named attribute. The two agree here only because `TrackId` is a primary key and cannot be NULL. The notation also exposes a deeper mismatch: pure relational algebra is defined over *sets*, so a strict reading would deduplicate and undercount. The γ operator is an extension to the algebra precisely because SQL aggregation requires multiset semantics.

Q14 — Which artist has the most albums?

```
SELECT T2.NAME FROM ALBUM AS T1
JOIN ARTIST AS T2 ON T1.ArtistId = T2.ArtistId
GROUP BY T2.Name ORDER BY COUNT(*) DESC LIMIT 1
```

$$\begin{aligned} A &\leftarrow \gamma_{ArtistId,\, Name;\ \text{COUNT}(AlbumId) \to AlbumCount}(\text{Artist} \bowtie_{\text{ArtistId}} \text{Album}) \\ M &\leftarrow \gamma_{\text{MAX}(AlbumCount) \to MaxCount}(A) \\ \text{Result} &\leftarrow \pi_{Name}\big(\sigma_{AlbumCount = MaxCount}(A \times M)\big) \end{aligned}$$

Comparison. This is the clearest case of the notation exposing what the SQL conceals. Relational algebra has no LIMIT, so "the most" must be expressed as a self-comparison against the maximum: aggregate, take the maximum, join back. Writing it that way reveals a real semantic difference. The algebraic form returns *every* artist tied for the maximum, whereas ORDER BY ... LIMIT 1 returns exactly one row and silently selects an arbitrary winner among ties. Our gold answer, Iron Maiden with 21 albums, is unique in this data, so the two forms agree — but only by accident of the data, not by equivalence of the queries. Grouping by `Name` rather than `ArtistId` carries the same class of hazard: two distinct artists sharing a name would be conflated.

Q17 — Count artists who have not released an album

```
SELECT COUNT(*) FROM ARTIST AS T1
WHERE NOT EXISTS (SELECT 1 FROM ALBUM AS T2 WHERE T2.ArtistId = T1.ArtistId)
```

$$\gamma_{\text{COUNT}(ArtistId)}(\pi_{ArtistId}(\text{Artist}) - \pi_{ArtistId}(\text{Album}))$$

Tuple relational calculus:

$$\{\, t \mid \exists a \in \text{Artist}\ (t.ArtistId = a.ArtistId\ \wedge\ \neg\exists b \in \text{Album}\,(b.ArtistId = a.ArtistId))\,\}$$

Comparison. The SQL expresses absence through a correlated subquery, which reads procedurally: for each artist, go look for an album. The algebra states the same thing as one antijoin, a set difference over projected keys, making it evident that this is a set operation on `ArtistId` values and nothing more. The calculus form is the most literal of the three, mapping NOT EXISTS directly onto $\neg\exists$. The formal notation also explains why we changed the original Spider gold SQL from NOT IN to NOT EXISTS: set difference is well defined over NULL-free key sets, but SQL's NOT IN evaluates to UNKNOWN rather than TRUE if the inner result contains a single NULL, which would make the count silently wrong. Three-valued logic has no counterpart in the algebra, so the notation hides a hazard that the SQL must handle deliberately.

Q02 — Customer last names with no invoice total above 20

```
SELECT LastName FROM CUSTOMER
EXCEPT
SELECT T1.LastName FROM CUSTOMER AS T1
JOIN Invoice AS T2 ON T1.CustomerId = T2.CustomerId WHERE T2.total > 20
```

$$\pi_{LastName}(\text{Customer}) - \pi_{LastName}(\sigma_{Total>20}(\text{Customer} \bowtie_{\text{CustomerId}} \text{Invoice}))$$

Comparison. Here SQL and algebra align almost symbol for symbol, since EXCEPT *is* the difference operator. What the notation makes explicit is the *level* at which the difference applies: both operands are projected down to `LastName`, so the operation subtracts *names*, not *customers*. A customer who happens to share a last name with a large spender would therefore be removed as well. Projecting before subtracting is not interchangeable with subtracting before projecting. In our data all 59 customers have distinct last names, so the result is correct — but the algebra shows that this correctness is a property of the data, not of the query. The notation likewise makes EXCEPT's implicit deduplication visible: set difference yields a set, which is why 55 names are returned rather than one row per qualifying customer.

5 Denormalized schema variant

Because Chinook arrives in 3NF, the variant in `sql/denormalized_schema.sql` deliberately reintroduces redundancy by collapsing the three join paths our benchmark questions traverse most often. Both versions coexist in `data/chinook_denorm.db`, so identical gold queries can run against either.

Flat table	Merges	Joins removed	Rows
`TrackFlat`	Track + Album + Artist + Genre + MediaType	4	3,503
`SalesFlat`	InvoiceLine + Invoice + Customer + Track	3	2,240
`CustomerFlat`	Customer + support-representative Employee	1	59

5.1 Dependencies and normal forms violated

TrackFlat (key `TrackId`). Since $TrackId \to AlbumId$ and $AlbumId \to AlbumTitle$, $ArtistId$, the dependency $TrackId \to AlbumTitle$ holds only transitively; likewise $ArtistId \to ArtistName$, $GenreId \to GenreName$ and $MediaTypeId \to MediaTypeName$. Non-key attributes thus depend on other non-key attributes, **violating 3NF**. The table remains in 2NF, as a single-attribute key admits no partial dependency. The redundancy is substantial: "Iron Maiden" is now stored 213 times and "U2" 135 times.

SalesFlat (key `InvoiceLineId`). Two dependency chains hold:

$$InvoiceLineId \to InvoiceId$$
$$InvoiceId \to InvoiceDate,\ BillingCity,\ BillingCountry,\ InvoiceTotal,\ CustomerId$$
$$CustomerId \to CustomerFirstName,\ CustomerLastName,\ CustomerAddress,\ CustomerCountry$$

Both are transitive, again **violating 3NF**. An invoice with fourteen line items repeats that customer's address fourteen times.

CustomerFlat (key `CustomerId`). The chain

$$CustomerId \to SupportRepId \to RepFirstName,\ RepLastName,\ RepTitle$$

is a transitive dependency, **violating 3NF**. With 59 customers served by three representatives, each representative's name is stored roughly twenty times.

5.2 Anomalies introduced

Update anomaly

Correcting an artist's name requires updating every track row for that artist (213 rows for Iron Maiden) rather than one row in `Artist`. A partial update leaves two spellings of the same artist with no way to determine which is authoritative.

Insert anomaly

`TrackFlat` cannot record an artist or genre that has no tracks, and `SalesFlat` cannot record a customer who has never been invoiced, as there is no row in which to place them without fabricating a track or line item. Note that Q17 asks precisely about artists with no albums, so it still requires the normalized `Artist` table.

Delete anomaly
: Deleting the final line item of an invoice erases that invoice's date and total; deleting an artist's last track erases the artist entirely.

Inconsistency risk
: `CustomerCountry` and `BillingCountry` now coexist in `SalesFlat` and may disagree, with no constraint available to prevent it.

5.3 Design decisions and cost

- Nullable foreign keys are flattened with LEFT JOIN. The present data contains no NULLs in those columns, but the schema permits them, and an INNER JOIN would silently drop rows, causing the denormalized copy to return different answers from the normalized one — which would invalidate the Deliverable 2 timing comparison.
- **Verified equivalence.** Six representative queries (Q07, Q10, Q11, Q14, Q20) and all table row counts return identical results on both schemas. Any timing difference measured in Deliverable 2 is therefore attributable to schema design, not to differing answers.
- **Storage cost.** The database grows from 984 KB to 1.7 MB, approximately 73% larger, for the same information content.
- The normalized tables are retained alongside the flat ones. That is itself a denormalization trade-off: duplicated storage in exchange for a join-free read path, which is the standard argument for a read-optimized reporting table.

6 Known limitations

- The question set contains easy and medium difficulty only, following the project guide's recommendation; no hard Spider questions are included.
- Q13 and Q14 use ORDER BY ... LIMIT 1, which returns an arbitrary row when values tie. No ties exist in this data, and §4 documents the consequence.
- Q02 subtracts last names rather than customers, which yields the correct answer only because all 59 customers have distinct last names.
- Chinook ships 11 foreign-key indexes of its own. An initial timing run showed normalized queries gaining almost nothing from added indexes, because the "unindexed" baseline was in fact already using `IFK_TrackGenreId`. The Deliverable 2 benchmark therefore drops and recreates Chinook's own indexes by name to obtain a genuinely unindexed baseline. We record this here because it affects the validity of any index comparison on this schema.
- The spelling errors inherited from Spider's question wording were retained deliberately (§3).

A Normalized schema (DDL)

Generated from the live database by `scripts/setup_db.py`; this is the exact text supplied to both models in the Deliverable 2 prompt.

```
CREATE TABLE [Album]
(
    [AlbumId] INTEGER NOT NULL,
    [Title] NVARCHAR(160) NOT NULL,
    [ArtistId] INTEGER NOT NULL,
    CONSTRAINT [PK_Album] PRIMARY KEY ([AlbumId]),
    FOREIGN KEY ([ArtistId]) REFERENCES [Artist] ([ArtistId])
                ON DELETE NO ACTION ON UPDATE NO ACTION
);

CREATE TABLE [Artist]
(
    [ArtistId] INTEGER NOT NULL,
    [Name] NVARCHAR(120),
    CONSTRAINT [PK_Artist] PRIMARY KEY ([ArtistId])
);

CREATE TABLE [Customer]
(
    [CustomerId] INTEGER NOT NULL,
    [FirstName] NVARCHAR(40) NOT NULL,
    [LastName] NVARCHAR(20) NOT NULL,
    [Company] NVARCHAR(80),
    [Address] NVARCHAR(70),
    [City] NVARCHAR(40),
    [State] NVARCHAR(40),
    [Country] NVARCHAR(40),
    [PostalCode] NVARCHAR(10),
    [Phone] NVARCHAR(24),
    [Fax] NVARCHAR(24),
    [Email] NVARCHAR(60) NOT NULL,
    [SupportRepId] INTEGER,
    CONSTRAINT [PK_Customer] PRIMARY KEY ([CustomerId]),
    FOREIGN KEY ([SupportRepId]) REFERENCES [Employee] ([EmployeeId])
                ON DELETE NO ACTION ON UPDATE NO ACTION
);

CREATE TABLE [Employee]
(
    [EmployeeId] INTEGER NOT NULL,
    [LastName] NVARCHAR(20) NOT NULL,
    [FirstName] NVARCHAR(20) NOT NULL,
    [Title] NVARCHAR(30),
    [ReportsTo] INTEGER,
    [BirthDate] DATETIME,
    [HireDate] DATETIME,
    [Address] NVARCHAR(70),
    [City] NVARCHAR(40),
    [State] NVARCHAR(40),
    [Country] NVARCHAR(40),
    [PostalCode] NVARCHAR(10),
    [Phone] NVARCHAR(24),
    [Fax] NVARCHAR(24),
    [Email] NVARCHAR(60),
    CONSTRAINT [PK_Employee] PRIMARY KEY ([EmployeeId]),
    FOREIGN KEY ([ReportsTo]) REFERENCES [Employee] ([EmployeeId])
                ON DELETE NO ACTION ON UPDATE NO ACTION
);
```

```
CREATE TABLE [Genre]
(
    [GenreId] INTEGER NOT NULL,
    [Name] NVARCHAR(120),
    CONSTRAINT [PK_Genre] PRIMARY KEY ([GenreId])
);

CREATE TABLE [Invoice]
(
    [InvoiceId] INTEGER NOT NULL,
    [CustomerId] INTEGER NOT NULL,
    [InvoiceDate] DATETIME NOT NULL,
    [BillingAddress] NVARCHAR(70),
    [BillingCity] NVARCHAR(40),
    [BillingState] NVARCHAR(40),
    [BillingCountry] NVARCHAR(40),
    [BillingPostalCode] NVARCHAR(10),
    [Total] NUMERIC(10,2) NOT NULL,
    CONSTRAINT [PK_Invoice] PRIMARY KEY ([InvoiceId]),
    FOREIGN KEY ([CustomerId]) REFERENCES [Customer] ([CustomerId])
                ON DELETE NO ACTION ON UPDATE NO ACTION
);

CREATE TABLE [InvoiceLine]
(
    [InvoiceLineId] INTEGER NOT NULL,
    [InvoiceId] INTEGER NOT NULL,
    [TrackId] INTEGER NOT NULL,
    [UnitPrice] NUMERIC(10,2) NOT NULL,
    [Quantity] INTEGER NOT NULL,
    CONSTRAINT [PK_InvoiceLine] PRIMARY KEY ([InvoiceLineId]),
    FOREIGN KEY ([InvoiceId]) REFERENCES [Invoice] ([InvoiceId])
                ON DELETE NO ACTION ON UPDATE NO ACTION,
    FOREIGN KEY ([TrackId]) REFERENCES [Track] ([TrackId])
                ON DELETE NO ACTION ON UPDATE NO ACTION
);

CREATE TABLE [MediaType]
(
    [MediaTypeId] INTEGER NOT NULL,
    [Name] NVARCHAR(120),
    CONSTRAINT [PK_MediaType] PRIMARY KEY ([MediaTypeId])
);

CREATE TABLE [Playlist]
(
    [PlaylistId] INTEGER NOT NULL,
    [Name] NVARCHAR(120),
    CONSTRAINT [PK_Playlist] PRIMARY KEY ([PlaylistId])
);

CREATE TABLE [PlaylistTrack]
(
    [PlaylistId] INTEGER NOT NULL,
    [TrackId] INTEGER NOT NULL,
    CONSTRAINT [PK_PlaylistTrack] PRIMARY KEY ([PlaylistId], [TrackId]),
    FOREIGN KEY ([PlaylistId]) REFERENCES [Playlist] ([PlaylistId])
                ON DELETE NO ACTION ON UPDATE NO ACTION,
```

```
    FOREIGN KEY ([TrackId]) REFERENCES [Track] ([TrackId])
                ON DELETE NO ACTION ON UPDATE NO ACTION
);

CREATE TABLE [Track]
(
    [TrackId] INTEGER NOT NULL,
    [Name] NVARCHAR(200) NOT NULL,
    [AlbumId] INTEGER,
    [MediaTypeId] INTEGER NOT NULL,
    [GenreId] INTEGER,
    [Composer] NVARCHAR(220),
    [Milliseconds] INTEGER NOT NULL,
    [Bytes] INTEGER,
    [UnitPrice] NUMERIC(10,2) NOT NULL,
    CONSTRAINT [PK_Track] PRIMARY KEY ([TrackId]),
    FOREIGN KEY ([AlbumId]) REFERENCES [Album] ([AlbumId])
                ON DELETE NO ACTION ON UPDATE NO ACTION,
    FOREIGN KEY ([GenreId]) REFERENCES [Genre] ([GenreId])
                ON DELETE NO ACTION ON UPDATE NO ACTION,
    FOREIGN KEY ([MediaTypeId]) REFERENCES [MediaType] ([MediaTypeId])
                ON DELETE NO ACTION ON UPDATE NO ACTION
);
```

B Questions, gold SQL, and verified results

Each entry gives the natural-language question as sent to the models, its verified gold SQL, and the result that SQL returns against `data/chinook.db`. Owner initials identify the team member who verified the answer.

Q01 — Find the average unite price for a track.

Difficulty	easy
Shape	lookup
Source	chinnok_1, train_spider.json index 830
Verified by	JD
Row order matters	no

```
SELECT AVG(UnitPrice) FROM Track
```

Result (1 row): (1.051)

Q02 — Find all the customer last names that do nout have invoice totals larger than 20.

Difficulty	medium
Shape	lookup
Source	chinnok_1, train_spider.json index 857
Verified by	IW
Row order matters	no

```
SELECT LastName FROM CUSTOMER EXCEPT SELECT T1.LastName FROM CUSTOMER AS T1 JOIN Invoice AS T2 ON
    T1.CustomerId = T2.CustomerId WHERE T2.total > 20
```

Result (55 rows): (Almeida), (Barnett), (Bernard), (Brooks), (Brown), (Chase), (Dubois), (Fernandes), (Francis), (Girard), (Gonçalves), (Gordon), (Goyer), (Gray), (Gruber), (Gutiérrez), (Hansen), (Harris), (Hughes), (Hämäläinen), ...

Q03 — What are the distinct billing countries of the invoices?

Difficulty	easy
Shape	lookup
Source	chinnok_1, train_spider.json index 815
Verified by	MA
Row order matters	no

```
SELECT DISTINCT BILLINGCOUNTRY FROM INVOICE
```

Result (24 rows): (Germany), (Norway), (Belgium), (Canada), (USA), (France), (Ireland), (United Kingdom), (Australia), (Chile), (India), (Brazil), (Portugal), (Netherlands), (Spain), (Sweden), (Czech Republic), (Finland), (Denmark), ...

Q04 — How many distinct cities does the employee live in?

Difficulty	easy
Shape	lookup
Source	chinnok_1, train_spider.json index 853
Verified by	JS
Row order matters	no

```
SELECT COUNT(DISTINCT city) FROM EMPLOYEE
```

Result (1 row): (3)

Q05 — List the phone numbers of all employees.

Difficulty	easy
Shape	lookup
Source	chinnok_1, train_spider.json index 864
Verified by	JD
Row order matters	no

```
SELECT Phone FROM EMPLOYEE
```

Result (8 rows): (+1 (780) 428-9482), (+1 (403) 262-3443), (+1 (403) 262-3443), (+1 (403) 263-4423), (1 (780) 836-9987), (+1 (403) 246-9887), (+1 (403) 456-9986), (+1 (403) 467-3351)

Q06 — What are the invoice dates for customers with the first name Astrid and the last name Gruber?

Difficulty	medium
Shape	join
Source	chinnok_1, train_spider.json index 856
Verified by	IW
Row order matters	no

```
SELECT T2.InvoiceDate FROM CUSTOMER AS T1 JOIN INVOICE AS T2 ON T1.CustomerId = T2.CustomerId
    WHERE T1.FirstName ='Astrid' AND T1.LastName = 'Gruber'
```

Result (7 rows): (2021-12-08 00:00:00), (2022-01-18 00:00:00), (2022-09-18 00:00:00), (2024-04-24 00:00:00), (2024-07-27 00:00:00), (2024-10-29 00:00:00), (2025-06-19 00:00:00)

Q07 — What are the addresses of customers living in Germany who have had an invoice?

Difficulty	medium
Shape	join
Source	chinnok_1, train_spider.json index 862
Verified by	MA
Row order matters	no

```
SELECT DISTINCT T1.Address FROM CUSTOMER AS T1 JOIN INVOICE AS T2 ON T1.CustomerId = T2.CustomerId
     WHERE T1.Country = 'Germany'
```

Result (4 rows): (Theodor-Heuss-StraSSe 34), (TauentzienstraSSe 8), (Berger StraSSe 10), (BarbarossastraSSe 19)

Q08 — Find the number of albums by that artist Metallica

Difficulty	medium
Shape	join
Source	chinnok_1, chinook_1, train_spider.json index 822
Verified by	JS
Row order matters	no

```
SELECT COUNT(*) FROM ALBUM AS T1 JOIN ARTIST AS T2 ON T1.ArtistId = T2.ArtistId WHERE T2.Name = '
    Metallica'
```

Result (1 row): (10)

Q09 — What are the duration of the longest and shortest pop tracks in milliseconds?

Difficulty	medium
Shape	join
Source	chinnok_1, train_spider.json index 881
Verified by	JD
Row order matters	no

```
SELECT MAX(T2.Milliseconds), MIN(T2.Milliseconds) FROM GENRE AS T1 JOIN TRACK AS T2 ON T1.GenreId
    = T2.GenreId WHERE T1.Name = 'Pop';
```

Result (1 row): (663426, 129666)

Q10 — Count the number of tracks that are of the media type 'AAC audio file'.

Difficulty	medium
Shape	join
Source	chinnok_1, train_spider.json index 866
Verified by	IW
Row order matters	no

```
SELECT COUNT(*) FROM MEDIATYPE AS T1 JOIN TRACK AS T2 ON T1.MediaTypeId = T2.MediaTypeId WHERE T1.
    Name = 'AAC audio file';
```

Result (1 row): (11)

Q11 — What is the first name and last name employee helps the customer with first name Leonie?

Difficulty	medium
Shape	join
Source	chinnok_1, train_spider.json index 849
Verified by	MA
Row order matters	no

```
SELECT T2.FirstName, T2.LastName FROM CUSTOMER AS T1 JOIN EMPLOYEE AS T2 ON T1.SupportRepId = T2.
    EmployeeId WHERE T1.FirstName = 'Leonie';
```

Result (1 row): (Steve, Johnson)
Adaptation: none needed chinook uses identical table/column names

Q12 — What are the first names and support rep ids for employees serving 10 or more customers?

Difficulty	medium
Shape	aggregation
Source	chinnok_1, train_spider.json index 870
Verified by	JS
Row order matters	no

```
SELECT T2.FirstName, T1.SupportRepId FROM CUSTOMER AS T1 JOIN EMPLOYEE AS T2 ON T1.SupportRepId =
    T2.EmployeeId GROUP BY T1.SupportRepId, T2.FirstName HAVING COUNT(*) >= 10;
```

Result (3 rows): (Jane, 3), (Margaret, 4), (Steve, 5)
Adaptation: corrected original gold SQL it selected customer's firstName instead of employee's and wasn't grouped correctly

Q13 — What is the least commone media type in all tracks?

Difficulty	medium
Shape	aggregation
Source	chinnok_1, train_spider.json index 837
Verified by	JD
Row order matters	yes

```
SELECT T1.Name FROM MEDIATYPE AS T1 JOIN TRACK AS T2 ON T1.MediaTypeId = T2.MediaTypeId GROUP BY
    T1.Name ORDER BY COUNT(*) ASC LIMIT 1;
```

Result (1 row): (Purchased AAC audio file)
Adaptation: tightened the GROUP BY to include both MediaTypeId and Name for correctness

Q14 — Which artist has the most albums?

Difficulty	medium
Shape	aggregation
Source	chinnok_1, train_spider.json index 825
Verified by	IW
Row order matters	yes

```
SELECT T2.NAME FROM ALBUM AS T1 JOIN ARTIST AS T2 ON T1.ArtistId = T2.ArtistId GROUP BY T2.Name
    ORDER BY COUNT(*) DESC LIMIT 1;
```

Result (1 row): (Iron Maiden)

Q15 — What are the titles and ids for albums containing tracks with unit price greater than 1?

Difficulty	medium
Shape	aggregation
Source	chinnok_1, train_spider.json index 840
Verified by	MA
Row order matters	no

```
SELECT T1.Title, T2.AlbumId FROM ALBUM AS T1 JOIN TRACK AS T2 ON T1.AlbumId = T2.AlbumId WHERE T2.
    UnitPrice > 1 GROUP BY T2.AlbumId, T1.Title;
```

Result (12 rows): (Battlestar Galactica: The Story So Far, 226), (Battlestar Galactica, Season 3, 227), (Heroes, Season 1, 228), (Lost, Season 3, 229), (Lost, Season 1, 230), (Lost, Season 2, 231), ...
Adaptation: added Title to the GROUP BY for correctness

Q16 — What are the last names of employees who serve at most 20 customers, including employees who serve no customers?

Difficulty	medium
Shape	aggregation
Source	chinnok_1, train_spider.json index 872
Verified by	JS
Row order matters	no

```
SELECT T2.LastName FROM EMPLOYEE AS T2 LEFT JOIN CUSTOMER AS T1 ON T1.SupportRepId = T2.EmployeeId
     GROUP BY T2.EmployeeId, T2.LastName HAVING COUNT(T1.CustomerId) <= 20;
```

Result (7 rows): (Adams), (Edwards), (Park), (Johnson), (Mitchell), (King), (Callahan)
Adaptation: corrected original gold SQL it selected customer's LastName instead of the employee's,

Q17 — Count the number of artists who have not realeased an album?

Difficulty	medium
Shape	subquery
Source	chinnok_1, train_spider.json index 888
Verified by	JD
Row order matters	no

```
SELECT COUNT(*) FROM ARTIST AS T1 WHERE NOT EXISTS (SELECT 1 FROM ALBUM AS T2 WHERE T2.ArtistId =
    T1.ArtistId);
```

Result (1 row): (71)
Adaptation: used NOT EXISTS instead of original NOT IN for NULL

Q18 — How many albums are there?

Difficulty	easy
Shape	lookup
Source	chinnok_1, train_spider.json index 807
Verified by	IW
Row order matters	no

```
SELECT COUNT(*) FROM ALBUM
```

Result (1 row): (347)
Adaptation: none needed chinook uses identical table/column names

Q19 — How many customers have email that contains 'gmail.com'?

Difficulty	easy
Shape	lookup
Source	chinnok_1, train_spider.json index 847
Verified by	MA
Row order matters	no

```
SELECT COUNT(*) FROM CUSTOMER WHERE Email LIKE '%gmail.com%'
```

Result (1 row): (8)
Adaptation: none needed chinook uses identical table/column names

Q20 — What is the average unit price of tracks that billing to Jazz genre?

Difficulty	medium
Shape	join
Source	chinnok_1, train_spider.json index 843
Verified by	JS
Row order matters	no

```
SELECT AVG(UnitPrice) FROM GENRE AS T1 JOIN TRACK AS T2 ON T1.GenreId = T2.GenreId WHERE T1.Name =
    'Jazz';
```

Result (1 row): (0.99)
Adaptation: none needed chinook uses identical table/column names